



Boosting OpenSearch Performance: Lucene Bulk Collection and gRPC Search in Action

Sakshi Nasha, Abdul Muneer

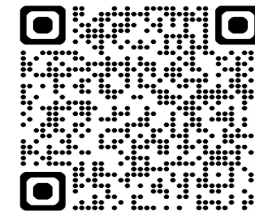


About us



Sakshi Nasha

-  Software Engineer @Cohesity
-  Open Source Contributor
-  25+ International talks on Go, Security, PostgreSQL & open-source technologies
- 



Abdul Muneer

- Open Source Developer @ NetApp Instaclustr
- Maintainer: OpenSearch MLCommons
- Committer: OpenJDK



Why this talk matters

- Modern search systems face two major bottlenecks:
Compute overhead + Transport overhead
- Opensearch: powerhouse for massive analytics -> requires aggregation which are **CPU intensive**
- JVM hot paths matter : it boils down to the code that is getting executed.
Can we reduce JVM execution overhead?
- API communication:
 - Time spent for every search request
= Time (Querying Lucene + moving data between client and OpenSearch)
 - **Can we reduce API communication latency?**



**“At scale,
even tiny
inefficiencies
compound into
massive performance
& cost impacts.”**

Lucene Bulk Collection

- Aggregations are CPU-heavy in analytics workloads.
- Traditional per-doc collection repeats expensive call patterns.
- The optimization theme: **do more work per call**, fewer dispatches.
- Collector and doc-value access are on hot path.
- Sub-aggregations multiply collection cost.
- **Up to 40%** performance improvement



Types of Aggregations

- Metric aggregations: avg/sum/stats
- Bucket aggregations: range/histogram/date_histogram/filters
- Pipeline aggregations: max_bucket, min_bucket



JVM concepts

- Virtual dispatch is expensive at scale.
- Method inlining is easier when call sites are predictable.
- Batching reduces repeated dispatch and helps JIT optimize.



Method Inline

```
public class BulkCollectionInliningDemo {  
    public static void main(String[] args) {  
        NumericSumCollector collector = new NumericSumCollector();  
        for (int doc = 0; doc < 1_000_000; doc++) {  
            long value = readDocValue(doc);  
            long running = collector.getRunningSum(); // JIT may inline  
            collector.setRunningSum(running + value); // JIT may inline  
        }  
        System.out.println("Collected sum: " + collector.getRunningSum());  
    }  
    static long readDocValue(int doc) { return doc % 7; }  
}  
  
class NumericSumCollector {  
    private long runningSum;  
    public long getRunningSum() { return runningSum; }  
    public void setRunningSum(long sum) { this.runningSum = sum; }  
}
```



Method Inline

```
public class BulkCollectionInliningDemo {  
  
    public static void main(String[] args) {  
        NumericSumCollector collector = new NumericSumCollector();  
        for (int doc = 0; doc < 1_000_000; doc++) {  
            long value = readDocValue(doc);  
            long running = collector.runningSum; // JIT may inline  
            collector.runningSum = (running + value); // JIT may inline  
        }  
        System.out.println("Collected sum: " + collector.getRunningSum());  
    }  
    static long readDocValue(int doc) { return doc % 7; }  
}  
  
class NumericSumCollector {  
    private long runningSum;  
    public long getRunningSum() { return runningSum; }  
    public void setRunningSum(long sum) { this.runningSum = sum; }  
}
```



Method Inline

```
29 17 4 BulkCollectionInliningDemo::readDocValue (6 bytes)
29 14 1 NumericSumCollector::getRunningSum (5 bytes)
29 15 1 NumericSumCollector::setRunningSum (6 bytes)
29 16 3 BulkCollectionInliningDemo::readDocValue (6 bytes) made not entrant: not
31 18 % 3 BulkCollectionInliningDemo::main @ 10 (57 bytes)
    @ 4 NumericSumCollector::<init> (5 bytes) inline
    @ 1 java.lang.Object::<init> (1 bytes) inline
    @ 17 BulkCollectionInliningDemo::readDocValue (6 bytes) inline
    @ 22 NumericSumCollector::getRunningSum (5 bytes) inline
    @ 32 NumericSumCollector::setRunningSum (6 bytes) inline
    @ 45 NumericSumCollector::getRunningSum (5 bytes) inline
    @ 48 java/lang/invoke/MethodHandle::invokeBasic (not loaded) fa:
    @ 53 java/io/PrintStream::println (not loaded) failed to inline:
31 19 3 BulkCollectionInliningDemo::main (57 bytes)
    @ 4 NumericSumCollector::<init> (5 bytes) inline
    @ 1 java.lang.Object::<init> (1 bytes) inline
    @ 17 BulkCollectionInliningDemo::readDocValue (6 bytes) inline
    @ 22 NumericSumCollector::getRunningSum (5 bytes) inline
    @ 32 NumericSumCollector::setRunningSum (6 bytes) inline
    @ 45 NumericSumCollector::getRunningSum (5 bytes) inline
    @ 48 java/lang/invoke/MethodHandle::invokeBasic (not loaded) fa:
    @ 53 java/io/PrintStream::println (not loaded) failed to inline:
32 20 % 4 BulkCollectionInliningDemo::main @ 10 (57 bytes)
33 18 % 3 BulkCollectionInliningDemo::main @ 10 (57 bytes) made not entrant: OSR in
```



Virtual Inline

```
/** Base collector - polymorphic entry point (virtual dispatch). */
abstract class MetricCollector {
    abstract void collect(long docValue);
}

/** Sum aggregation partial collector. */
class SumCollector extends MetricCollector {
    private long partialSum;
    @Override
    void collect(long docValue) {
        // Simulate lightweight per-doc work
        partialSum += docValue;
    }
    long getPartialSum() { return partialSum; }
}

/** Max aggregation partial collector. */
class MaxCollector extends MetricCollector {
    private long partialMax = Long.MIN_VALUE;
    @Override
    void collect(long docValue) {
        if (docValue > partialMax) partialMax = docValue;
    }
    long getPartialMax() { return partialMax; }
}

/** Count aggregation partial collector. */
class CountCollector extends MetricCollector {
    private long docCount;
    @Override
    void collect(long docValue) {
        docCount++;
    }
    long getDocCount() { return docCount; }
}
```



Virtual Inline

```
public static void main(String[] args) {
    int totalDocs = 100_000_000; // large dataset for measurable difference
    List<MetricCollector> collectors = new ArrayList<>(totalDocs);
    // Mixed metric collectors – like heterogeneous agg pipeline per doc
    for (int i = 0; i < totalDocs; i++) {
        if (i % 3 == 0) collectors.add(new SumCollector());
        else if (i % 3 == 1) collectors.add(new MaxCollector());
        else collectors.add(new CountCollector());
    }
    // --- Per-doc polymorphic collection (virtual dispatch every time) ---
    long start = System.nanoTime();
    for (int doc = 0; doc < totalDocs; doc++) {
        long docValue = doc % 7; // pretend doc_values[doc]
        collectors.get(doc).collect(docValue); // virtual call per doc
    }
    long end = System.nanoTime();
    System.out.printf(
        "Per-doc polymorphic collect time: %.2f ms%n",
        (end - start) / 1_000_000.0
    );
}
```



Virtual Inline

```
// --- Group by collector type (bulk-friendly layout) ---
List<SumCollector> sumCollectors = new ArrayList<>();
List<MaxCollector> maxCollectors = new ArrayList<>();
List<CountCollector> countCollectors = new ArrayList<>();
for (MetricCollector c : collectors) {
    if (c instanceof SumCollector) sumCollectors.add((SumCollector) c);
    else if (c instanceof MaxCollector) maxCollectors.add((MaxCollector) c);
    else countCollectors.add((CountCollector) c);
}
// --- Grouped / monomorphic collection ---
start = System.nanoTime();
for (int i = 0; i < sumCollectors.size(); i++) {
    sumCollectors.get(i).collect(i % 7);
}
for (int i = 0; i < maxCollectors.size(); i++) {
    maxCollectors.get(i).collect(i % 7);
}
for (int i = 0; i < countCollectors.size(); i++) {
    countCollectors.get(i).collect(i % 7);
}
end = System.nanoTime();
System.out.printf(
    "Bulk grouped collect time: %.2f ms%n",
    (end - start) / 1_000_000.0
);
```



Key Lucene change #1: bulk doc-values retrieval

- Important code idea introduced

```
// Before: advanceExact + longValue per doc  
// After: bulk retrieval  
values.longValues(count, docBuffer, valueBuffer, missingSentinel);
```

- **Why this helps:**
- Fewer API calls for same number of docs
- Better cache-local processing over arrays
- Lower per-doc overhead



Key Lucene change #1: bulk doc-values retrieval

```
81 + public void longValues(int size, int[] docs, long[] values, long defaultValue)
82 +     throws IOException {
83 +     for (int i = 0; i < size; ++i) {
84 +         int doc = docs[i];
85 +         long value;
86 +         if (advanceExact(doc)) {
87 +             value = longValue();
88 +         } else {
89 +             value = defaultValue;
90 +         }
91 +         values[i] = value;
92 +     }
93 + }
```

```
163 -     for (int i = 0; i < size; ++i) {
164 -         if (norms.advanceExact(buffer.docs[i])) {
165 -             normValues[i] = norms.longValue();
166 -         } else {
167 -             normValues[i] = 1L;
168 -         }
169 -     }
```



Key Lucene change #2: batched collector input

- Important code idea introduced

```
for (int count = stream.toArray(docBuffer);  
    count != 0;  
    count = stream.toArray(docBuffer)) {  
  
    values.longValues(count, docBuffer, valueBuffer, Long.MIN_VALUE);  
    // local tight loop for aggregation  
}
```

- **Why this helps:**
- Replaces repeated single-doc collection with chunked processing
- Less virtual dispatch pressure in hot loop
- Better inlining opportunities



Key Lucene change #3: range-oriented collection path

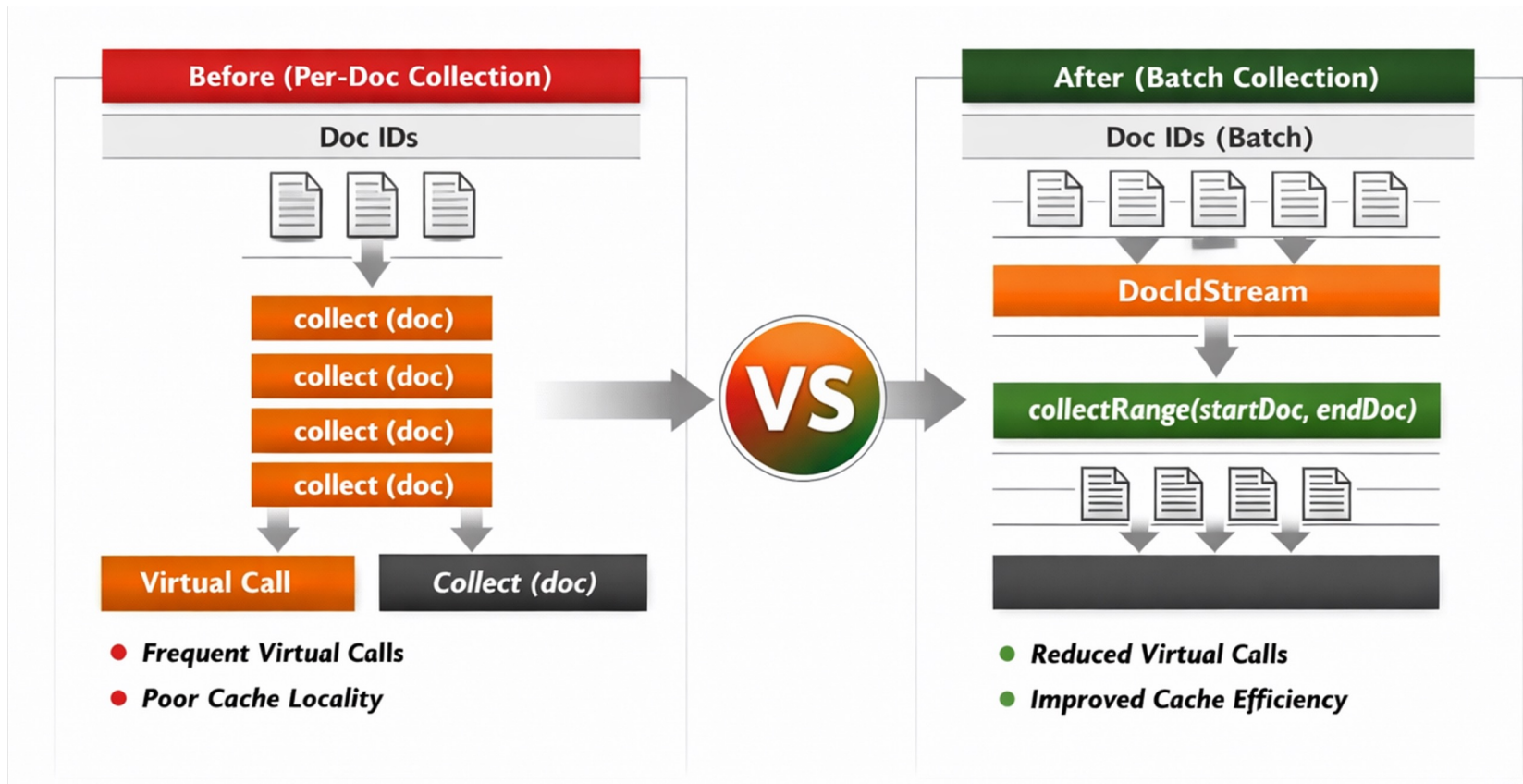
- Important code idea introduced

```
collector.collectRange(minDoc, maxDoc);
```

- **Why this helps:**
- Reduces Virtual Dispatch Overhead
- Improves CPU Efficiency
- Enhances JIT Optimization



Key Lucene change #3: range-oriented collection path



Let's move to
reducing **API**
communication
latency!



Search Latency Is More Than Search Execution



Search Execution

- Query parsing
- Lucene execution
- Aggregation processing

Communication Overhead

- JSON serialization
- HTTP request/response handling
- Network transfer
- Client-side parsing

“Transport overhead becomes significant at scale”



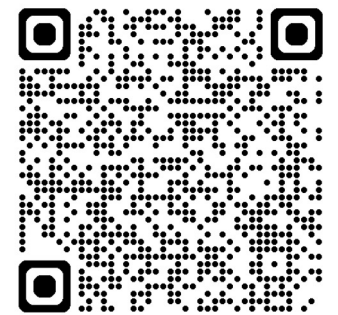
ASPECT	JSON (REST APIs)	PROTOBUF (gRPC)
READABILITY	 Human-readable text format <pre>{ "id": 123, "name": "Alice", "active": true }</pre>	 Binary format, not human-readable <pre>0A 07 41 6C 69 63 65 10 01</pre>
SCHEMA	 Schema-free / flexible	 Predefined schema contract (.proto) <pre>message User { int32 id = 1; string name = 2; bool active = 3; }</pre>
PAYLOAD STRUCTURE	Repeated field names included in payload <pre>{ "id": 123, "name": "Alice", "active": true }</pre>	Uses compact field numbers & wire types <pre>0A 07 41 6C 69 63 65 10 01</pre> field 1 (len) value ("Alice") field 2 (varint)
PAYLOAD SIZE	Larger payload size  ~150 bytes per message	Compact binary payload  ~40 bytes per message
NETWORK TRANSFER	More bytes transferred over network 	Fewer bytes transferred over network 
BANDWIDTH USAGE	 Higher network bandwidth consumption	 Lower network bandwidth consumption
CPU & RAM USAGE	More parsing overhead and memory usage 	Efficient serialization/deserialization 
LATENCY	 Higher latency	 Lower latency
PARSING SPEED	 Slower text parsing	 Faster binary parsing

JSON + REST

REST + PROTOBUF

PROTOBUF + gRPC

**Protobuf can also be
used with REST !!**



Why OpenSearch Added Native gRPC

- **Runs directly inside OpenSearch nodes**
- Not a proxy or gateway
- **Shares same request handlers as REST**
- Functional equivalence between REST and gRPC
- Incremental adoption possible



Keeping REST and gRPC in Sync

The Core Problem

Two APIs must always stay identical:

- Same functionality
- Same inputs
- Same outputs
- Same validation rules

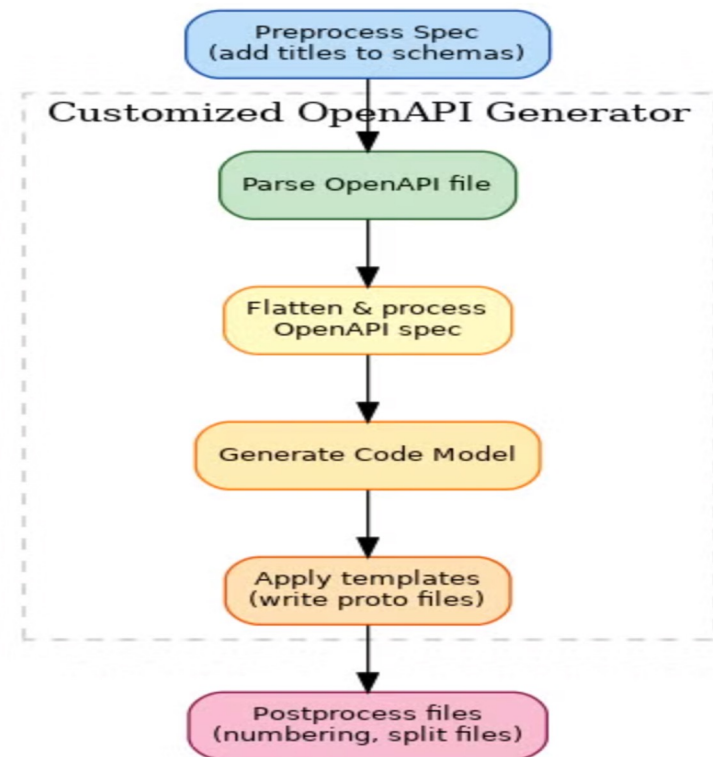
Manual synchronization does not scale.



Automated OpenAPI → Protobuf Pipeline

Pipeline Stages

1. Preprocessing
Normalize OpenAPI specs
2. Core Conversion
Generate protobuf schemas + gRPC services
3. Validation & Compatibility Checks
Ensure REST and gRPC stay equivalent



Where gRPC Stands today in OpenSearch

- gRPC support was introduced in OpenSearch **3.0**.
- **Bulk** and **k-NN** gRPC APIs generally available in **3.2**.
- **Search** gRPC API still experimental.
- OpenSearch Benchmark (OSB) supports this implementation
- OSB includes native gRPC workload configurations for performance testing.



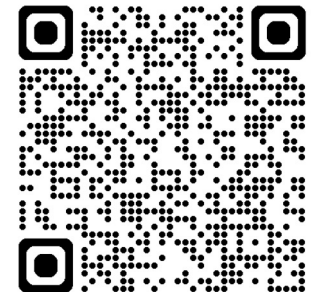
How to use gRPC in OpenSearch

```
C:\Users\snasha\OpenSearch\distribution\archives\windows-zip\build\install\opensearch-3.7.0-SNAPSHOT\config\opensearch.yml - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
opensearch.yml gRPC_benchmark.go gRPC_concurrent_benchmark.go rest_benchmark.go rest_concurrent_benchmark.go
102 #
103 #opensearch.experimental.feature.segment_replication_experimental.enabled: false
104 #
105 # Gates the functionality of a new parameter to the snapshot restore API
106 # that allows for creation of a new index type that searches a snapshot
107 # directly in a remote repository without restoring all index data to disk
108 # ahead of time.
109 #
110 #opensearch.experimental.feature.searchable_snapshot.enabled: false
111 #
112 #
113 # Gates the functionality of enabling extensions to work with OpenSearch.
114 # This feature enables applications to extend features of OpenSearch outside of
115 # the core.
116 #
117 #opensearch.experimental.feature.extensions.enabled: false
118 #
119 #
120 # Gates the optimization of datetime formatters caching along with change in default datetime formatter
121 # Once there is no observed impact on performance, this feature flag can be removed.
122 #
123 #opensearch.experimental.optimization.datetime_formatter_caching.enabled: false
124 aux.transport.types: [transport-grpc]
125 aux.transport.transport-grpc.port: 9400-9500
126 grpc.host: 0.0.0.0
127 grpc.bind_host: 0.0.0.0
128 grpc.publish_host: 127.0.0.1
129 grpc.publish_port: 9400
```



How to use gRPC in OpenSearch

```
Select Command Prompt - bin\opensearch.bat
builderProtoConverter(s) to register
[2026-05-15T08:59:44,303][INFO ][o.o.t.g.GrpcPlugin          ] [VRASNWIN2156] No external AggregatePro
toConverter(s) to register
[2026-05-15T08:59:44,746][INFO ][o.o.t.NettyAllocator      ] [VRASNWIN2156] creating NettyAllocator
with the following configs: [name=opensearch_configured, chunk_size=512kb, suggested_max_allocation
_size=512kb, factors={opensearch.unsafe.use_netty_default_chunk_and_page_size=false, glgc_enabled=t
rue, glgc_region_size=2mb}]
[2026-05-15T08:59:44,823][INFO ][o.o.d.DiscoveryModule     ] [VRASNWIN2156] using discovery type [ze
n] and seed hosts providers [settings]
[2026-05-15T08:59:45,159][WARN ][o.o.g.DanglingIndicesState] [VRASNWIN2156] gateway.auto_import_dan
gling_indices is disabled, dangling indices will not be automatically detected or imported and must
be managed manually
[2026-05-15T08:59:45,429][INFO ][o.o.n.Node                  ] [VRASNWIN2156] initialized
[2026-05-15T08:59:45,430][INFO ][o.o.n.Node                  ] [VRASNWIN2156] starting ...
[2026-05-15T08:59:45,783][INFO ][o.o.t.g.Netty4GrpcServerTransport] [VRASNWIN2156] publish_address
{127.0.0.1:9400}, bound_addresses {0.0.0.0:9400}
[2026-05-15T08:59:45,783][INFO ][o.o.t.g.Netty4GrpcServerTransport] [VRASNWIN2156] Started gRPC ser
ver on port PortsRange{portRange='9400-9500'} with 8 executor threads
[2026-05-15T08:59:45,932][INFO ][o.o.t.TransportService    ] [VRASNWIN2156] publish_address {127.0.0
.1:9300}, bound_addresses {[::1]:9300}, {127.0.0.1:9300}
[2026-05-15T08:59:45,934][INFO ][o.o.t.TransportService    ] [VRASNWIN2156] Remote clusters initiali
zed successfully.
[2026-05-15T08:59:46,149][WARN ][o.o.b.BootstrapChecks       ] [VRASNWIN2156] initial heap size [10737
41824] not equal to maximum heap size [4294967296]; this can cause resize pauses
[2026-05-15T08:59:46,150][WARN ][o.o.b.BootstrapChecks       ] [VRASNWIN2156] the default discovery se
tings are unsuitable for production use; at least one of [discovery.seed_hosts, discovery.seed_pro
viders, cluster.initial_cluster_manager_nodes / cluster.initial_master_nodes] must be configured
[2026-05-15T08:59:46,164][INFO ][o.o.c.c.ClusterBootstrapService] [VRASNWIN2156] no discovery confi
```



Benchmark Results: REST vs gRPC

k-NN vector search

- **P50 latency** improvements of **4.74%**.
- **Mean throughput** increased by **17.24%**. allowing the same cluster to handle more queries per second.

The following table shows the latency comparison.

Percentile	REST (ms)	gRPC (ms)	Improvement
P50	5.33	5.08	4.74%
P90	6.00	5.64	5.98%

The following table shows the throughput comparison.

REST (ops/sec)	gRPC (ops/sec)	Improvement
143.26	173.12	17.24%



Benchmark Results: REST vs gRPC

Bulk ingestion

- **~22% latency reduction** across different bulk sizes
- **~16% payload reduction** because of binary encoding

Documents per request	gRPC throughput benefit	gRPC latency reduction (P50)
10,000	16.2%	22.6%
5,000	15.8%	22.4%
2,500	14.8%	21.1%
1,000	12%	13.6%

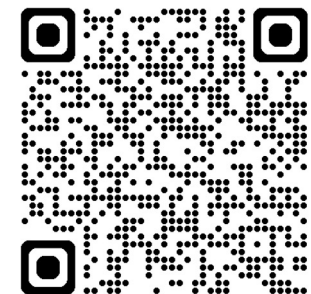


Local OpenSearch Experiment

Same: index, query, dataset, persistent connections

Final Benchmark Results

Method	Avg Latency
Persistent Go REST	~4.46 ms
Persistent Go gRPC	~3.66 ms



~18–22% lower latency with gRPC



Where the Performance Gains Come From

Benefits significantly from gRPC for the following reasons:

1. Compact payloads : **53% reduction** in **payload size**
 - a. Reduces network transfer time
 - b. Less parsing work : Lower CPU utilization on both client and server sides.
 - c. **58% reduction** in **client-side processing time** -> Faster serialization/deserialization
2. HTTP/2 **multiplexing** allows **concurrent** bulk requests over a single connection.



Where gRPC Fits Best

Best for:

- High-throughput services
- Internal microservices (node2node)
- Large payloads
- Bulk ingestion
- Vector search
- Service-to-service communication

REST still great for:

- Simplicity
- Debugging
- Broad compatibility
- External APIs



Final Takeaways

Lucene Bulk Collection:

- Up to **40% faster aggregations**

gRPC:

- Nearly **2x throughput in concurrent search**

“Hot-path optimization at every layer matters.”



Lucene Bulk Collection:

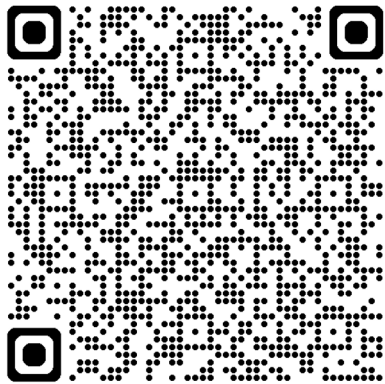
- <https://github.com/apache/lucene/pull/15149>
- <https://github.com/opensearch-project/OpenSearch/issues/19324>
- <https://github.com/apache/lucene/pull/15173>
- <https://github.com/apache/lucene/pull/14401>
- <https://github.com/opensearch-project/OpenSearch/pull/19933>

gRPC support

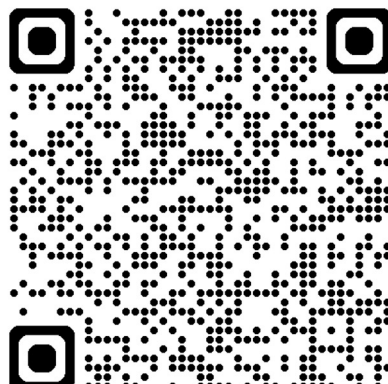
- My github repo : <https://github.com/wheresNasha/grpc>
- <https://github.com/opensearch-project/OpenSearch/issues/10684>



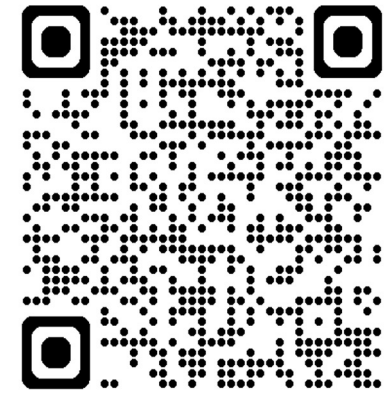
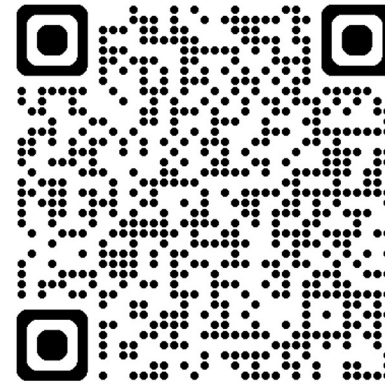
Future References



Project Roadmap



Documentation & Blogs

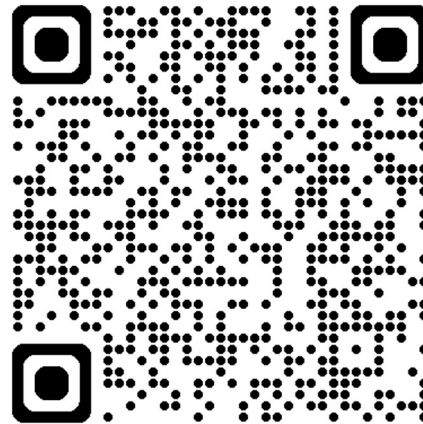


Call for Contributors

Are you ready to experiment grpc

We look forward to your contributions and hear the performance improvements!

Call for maintainers for go :





Q&A

Let's chat

Connect with us 

